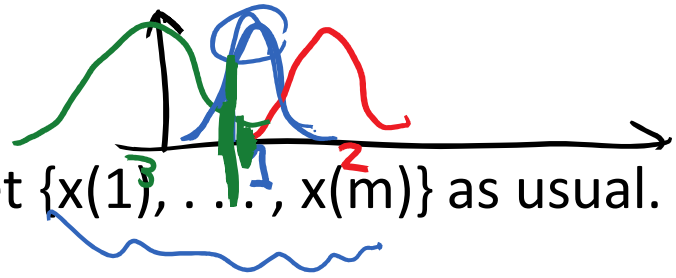
Mixtures of Gaussians and the EM algorithm

Mixture of Gaussians model



- Suppose that we are given a training set {x(1), . . . , x(m)} as usual.
- We wish to model the data by specifying a joint distribution

$$p\left(x^{(i)}, z^{(i)}\right) = p\left(x^{(i)} \mid z^{(i)}\right) p\left(z^{(i)}\right).$$

- Here, $z^{(i)} \sim Categorical\ (\phi)$ (where $\phi_j \geq 0, \sum_{j=1}^{k} \phi_j = 1$, and the parameter ϕ_j gives $p\left(z^{(i)} = j\right)$), and $x^{(i)} \mid z^{(i)} = j \sim \mathcal{N}\left(\mu_j, \Sigma_j\right)$.
- We let k denote the number of values that the $z^{(i)}$'s can take on.
- $z^{(i)}$'s are latent random variables, meaning that they're hidden/unobserved.

Mixture of Gaussians model

- The parameters of our model are thus ϕ, μ and Σ. To estimate them, we can write down the likelihood of our data:

$$\ell(\phi, \mu, \Sigma) = \sum_{i=1}^{m} \log p\left(x^{(i)}; \phi, \mu, \Sigma\right)$$

$$= \sum_{i=1}^{m} \log \sum_{z^{(i)}=1}^{k} p\left(x^{(i)} \mid z^{(i)}; \mu, \Sigma\right) p\left(z^{(i)}; \phi\right)$$

The random variables $z^{(i)}$ indicate which of the k Gaussians each $x^{(i)}$ had come from. **Note that if we knew what the $z^{(i)}$'s were, the maximum likelihood problem would have been easy.** شبيه حل GDA

Mixture of Gaussians model

we could then write down the likelihood as

$$\ell(\phi, \mu, \Sigma) = \sum_{i=1}^{m} \log p\left(x^{(i)} \mid z^{(i)}; \mu, \Sigma\right) + \log p\left(z^{(i)}; \phi\right).$$

Maximizing this with respect to ϕ, μ and Σ gives the parameters:

$$\begin{cases} \phi_j & = \frac{1}{m}\sum_{i=1}^{m} 1\{z^{(i)} = j\}, \\ \mu_j & = \frac{\sum_{i=1}^{m} 1\{z^{(i)} = j\} x^{(i)}}{\sum_{i=1}^{m} 1\{z^{(i)} = j\}}, \\ \Sigma_j & = \frac{\sum_{i=1}^{m} 1\{z^{(i)} = j\}(x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^{m} 1\{z^{(i)} = j\}}. \end{cases}$$

Mixture of Gaussians model (EM algorithm)

- In our density estimation problem, the $z^{(i)}$'s are not known. What can we do?

The EM algorithm is an iterative algorithm that has two main steps.

Initialize μ, Σ, ϕ randomly

1. E-step, it tries to "guess" the values of the $z^{(i)}$'s.

2. M-step, it updates the parameters of our model based on our guesses.

μ, Σ, ϕ

EM algorithm

Here's the algorithm:

Repeat until convergence: {

(E-step) For each i, j, set

$$w_j^{(i)} := p\left(z^{(i)} = j \mid x^{(i)}; \phi, \mu, \Sigma\right)$$

(M-step) Update the parameters:

$$\phi_j := \frac{1}{m}\sum_{i=1}^{m} w_j^{(i)},$$

$$\mu_j := \frac{\sum_{i=1}^{m} w_j^{(i)} x^{(i)}}{\sum_{i=1}^{m} w_j^{(i)}},$$

$$\Sigma_j := \frac{\sum_{i=1}^{m} w_j^{(i)} \left(x^{(i)} - \mu_j\right)\left(x^{(i)} - \mu_j\right)^T}{\sum_{i=1}^{m} w_j^{(i)}} \}$$

EM algorithm

In the E-step, we calculate the posterior probability of our parameters the $z^{(i)}$'s, given the $x^{(i)}$ and using the current setting of our parameters.

$$p\left(z^{(i)} = j \mid x^{(i)}; \phi, \mu, \Sigma\right) = \frac{p\left(x^{(i)} \mid z^{(i)} = j; \mu, \Sigma\right) p\left(z^{(i)} = j; \phi\right)}{\sum_{l=1}^{k} p(x^{(i)} \mid z^{(i)} = l; \mu, \Sigma) p(z^{(i)} = l; \phi)}$$

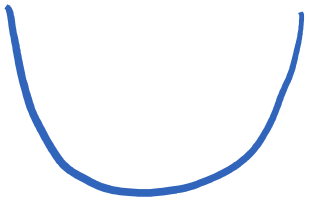
Here, $p\left(x^{(i)} \mid z^{(i)} = j; \mu, \Sigma\right)$ is given by evaluating the density of a Gaussian with mean μ_j and covariance Σ_j at $x^{(i)}$; $p\left(z^{(i)} = j; \phi\right)$ is given by ϕ_j, and so on.

Mixture of Gaussians model

- The EM-algorithm is also reminiscent of the K-means clustering algorithm, except that instead of the "hard" cluster assignments $c(i)$, we instead have the "soft" assignments $w_j^{(i)}$.
- Similar to K-means, it is also susceptible to local optima, so reinitializing at several different initial parameters may be a good idea.

EM algorithm

EM algorithm



Jensen's Inequality

Let f be a function whose domain is the set of real numbers. Recall that f is a convex function if $f''(x) \geq 0$ (for all $x \in \mathbb{R}$). In the case of f taking vector-valued inputs, this is generalized to the condition that its hessian H is positive semi-definite ($H \geq 0$). If $f''(x) > 0$ for all x, then we say f is **strictly** convex (in the vector-valued case, the corresponding statement is that H must be positive definite, written $H > 0$). Jensen's inequality can then be stated as follows:

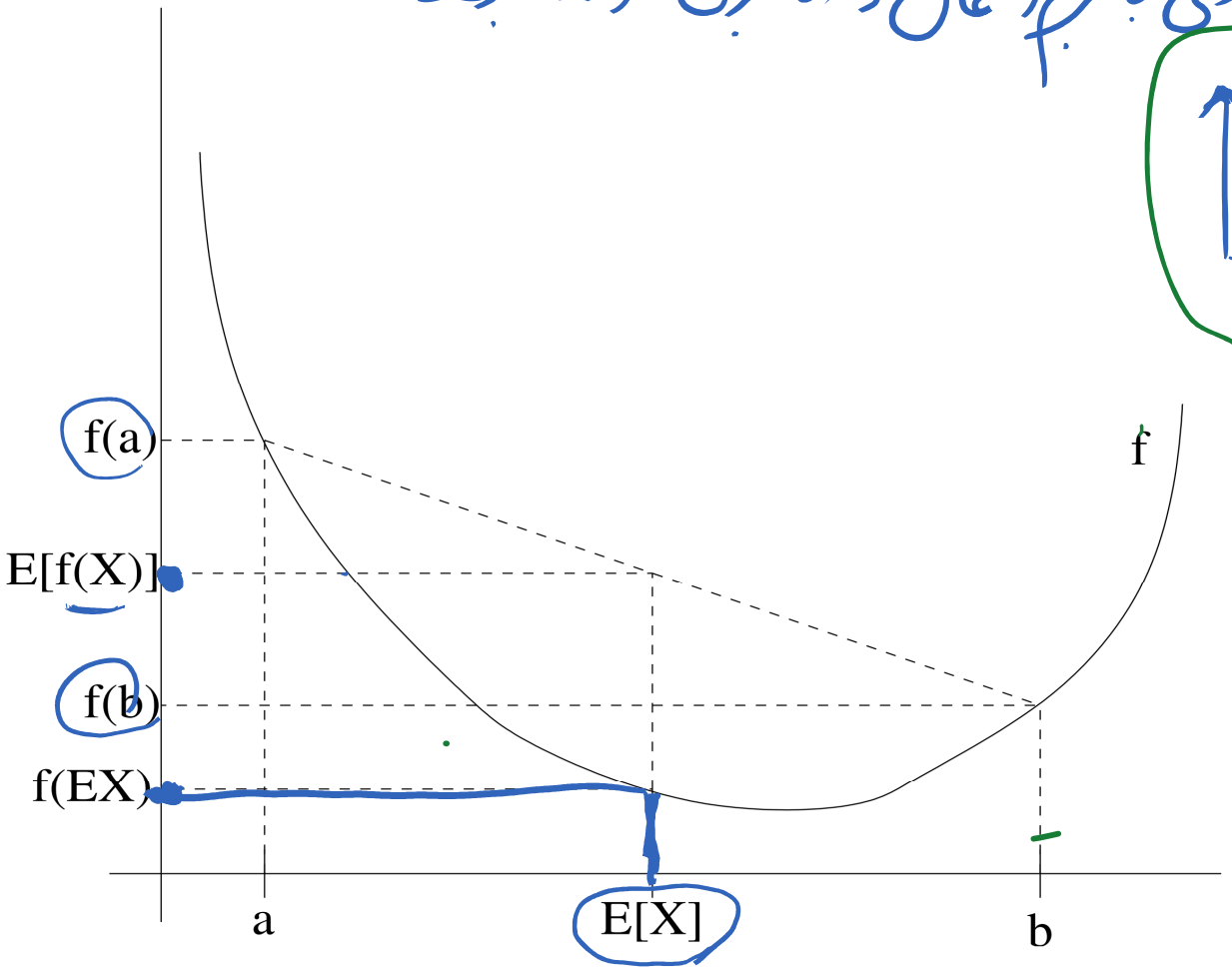
Theorem. Let f be a convex function, and let X be a random variable. Then:

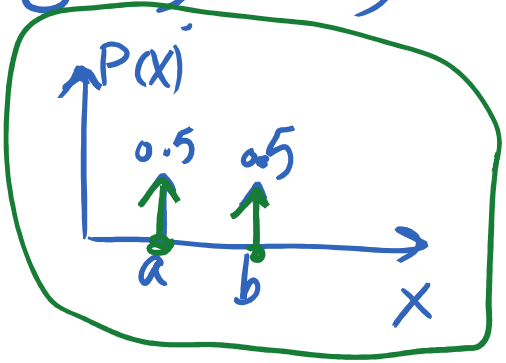
$$\mathrm{E}[f(X)] \geq f(\mathrm{E}X).$$

$E(X) = \int x\, p(x)\, dx$

Moreover, if f is strictly convex, then $\mathrm{E}[f(X)] = f(\mathrm{E}X)$ holds true if and only if $X = \mathrm{E}[X]$ with probability 1 (i.e., if X is a constant).

EM algorithm





Suppose we have an estimation problem in which we have a training set $\{x^{(1)}, \ldots, x^{(n)}\}$ consisting of n independent examples. We have a latent variable model $p(x, z; \theta)$ with z being the latent variable (which for simplicity is assumed to take finite number of values). The density for x can be obtained by marginalized over the latent variable z:

$$p(x; \theta) = \sum_z p(x, z; \theta) \tag{11.1}$$

$\int p(x, z; \theta)\, dz$ بیرون z را

We wish to fit the parameters θ by maximizing the log-likelihood of the data, defined by

$$\ell(\theta) = \sum_{i=1}^{n} \log p(x^{(i)}; \theta) \tag{11.2}$$

We can rewrite the objective in terms of the joint density $p(x, z; \theta)$ by

$$\ell(\theta) = \sum_{i=1}^{n} \log p(x^{(i)}; \theta) \tag{11.3}$$

$$= \sum_{i=1}^{n} \log \sum_{z^{(i)}} p(x^{(i)}, z^{(i)}; \theta). \tag{11.4}$$

max $\ell(\theta)$
non-convex Problem

EM algorithm

$l(\theta) \geq l_1$ حدهایی

In such a setting, the EM algorithm gives an efficient method for maximum likelihood estimation. Maximizing $\ell(\theta)$ explicitly might be difficult, and our strategy will be to instead repeatedly construct a lower-bound on ℓ (E-step), and then optimize that lower-bound (M-step).

It turns out that the summation $\sum_{i=1}^{n}$ is not essential here, and towards a simpler exposition of the EM algorithm, we will first consider optimizing the the likelihood $\log p(x)$ for **a single example** x. After we derive the algorithm for optimizing $\log p(x)$, we will convert it to an algorithm that works for n examples by adding back the sum to each of the relevant equations. Thus, now we aim to optimize $\log p(x;\theta)$ which can be rewritten as

برای یک نمونه آموزشی x

$$\log p(x;\theta) = \log \sum_{z} p(x,z;\theta) \tag{11.5}$$

EM algorithm

Let Q be a distribution over the possible values of z. That is, $\sum_z Q(z) = 1$, $Q(z) \geq 0$).

Consider the following:

$$\log p(x;\theta) = \log \sum_z p(x,z;\theta)$$

$$= \log \sum_z Q(z) \frac{p(x,z;\theta)}{Q(z)} \tag{11.6}$$

$$\geq \sum_z Q(z) \log \frac{p(x,z;\theta)}{Q(z)} \tag{11.7}$$

The last step of this derivation used Jensen's inequality. Specifically, $f(x) = \log x$ is a concave function, since $f''(x) = -1/x^2 < 0$ over its domain

$$\sum_z Q(z) \left[\frac{p(x, z; \theta)}{Q(z)} \right]$$

in the summation is just an expectation of the quantity $[p(x, z; \theta)/Q(z)]$ with respect to z drawn according to the distribution given by Q.[6] By Jensen's inequality, we have

$$f\left(\mathrm{E}_{z \sim Q} \left[\frac{p(x, z; \theta)}{Q(z)} \right] \right) \geq \mathrm{E}_{z \sim Q} \left[f\left(\frac{p(x, z; \theta)}{Q(z)} \right) \right],$$

where the "$z \sim Q$" subscripts above indicate that the expectations are with respect to z drawn from Q. This allowed us to go from Equation (11.6) to Equation (11.7).

Now, for **any** distribution Q, the formula (11.7) gives a lower-bound on $\log p(x; \theta)$. There are many possible choices for the Q's. Which should we choose? Well, if we have some current guess θ of the parameters, it seems natural to try to make the lower-bound tight at that value of θ. I.e., we will make the inequality above hold with equality at our particular value of θ.

To make the bound tight for a particular value of θ, we need for the step involving Jensen's inequality in our derivation above to hold with equality. For this to be true, we know it is sufficient that the expectation be taken over a "constant"-valued random variable. I.e., we require that

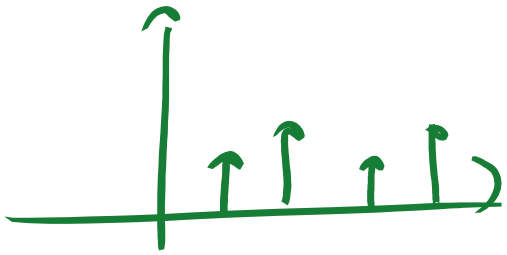
$$\frac{p(x, z; \theta)}{Q(z)} = c$$

for some constant c that does not depend on z. This is easily accomplished by choosing

$$Q(z) \propto p(x, z; \theta).$$

EM algorithm

Actually, since we know $\sum_z Q(z) = 1$ (because it is a distribution), this further tells us that



$$\begin{aligned} Q(z) &= \frac{p(x,z;\theta)}{\sum_z p(x,z;\theta)} \\ &= \frac{p(x,z;\theta)}{p(x;\theta)} \\ &= p(z|x;\theta) \end{aligned} \tag{11.8}$$

$Q(z) = p(z|x;\theta)$

EM algorithm

Thus, we simply set the Q's to be the posterior distribution of the z's given x and the setting of the parameters θ.

Indeed, we can directly verify that when $Q(z) = p(z|x;\theta)$, then equation (11.7) is an equality because

$$\sum_z Q(z) \log \frac{p(x,z;\theta)}{Q(z)} = \sum_z p(z|x;\theta) \log \frac{p(x,z;\theta)}{p(z|x;\theta)}$$

lower bound

$$= \sum_z p(z|x;\theta) \log \frac{p(z|x;\theta)p(x;\theta)}{p(z|x;\theta)}$$

$$= \sum_z p(z|x;\theta) \log p(x;\theta)$$

$$= \log p(x;\theta) \sum_z p(z|x;\theta)$$

$$= \log p(x;\theta) \qquad \text{(because } \textstyle\sum_z p(z|x;\theta) = 1\text{)}$$

For convenience, we call the expression in Equation (11.7) the **evidence lower bound** (ELBO) and we denote it by

$$\text{ELBO}(x; Q, \theta) = \sum_z Q(z) \log \frac{p(x, z; \theta)}{Q(z)} \tag{11.9}$$

With this equation, we can re-write equation (11.7) as

$$\forall Q, \theta, x, \quad \log p(x; \theta) \geq \text{ELBO}(x; Q, \theta) \tag{11.10}$$

Intuitively, the EM algorithm alternatively updates Q and θ by a) setting $Q(z) = p(z|x; \theta)$ following Equation (11.8) so that $\text{ELBO}(x; Q, \theta) = \log p(x; \theta)$ for x and the current θ, and b) maximizing $\text{ELBO}(x; Q, \theta)$ w.r.t θ while fixing the choice of Q.

Recall we have a training set $\{x^{(1)}, \ldots, x^{(n)}\}$. Note that the optimal choice of Q is $p(z|x;\theta)$, and it depends on the particular example x. Therefore here we will introduce n distributions $Q_1, \ldots, Q_n$, one for each example $x^{(i)}$. For each example $x^{(i)}$, we can build the evidence lower bound

$$\log p(x^{(i)};\theta) \geq \mathrm{ELBO}(x^{(i)}; Q_i, \theta) = \sum_{z^{(i)}} Q_i(z^{(i)}) \log \frac{p(x^{(i)}, z^{(i)};\theta)}{Q_i(z^{(i)})}$$

Taking sum over all the examples, we obtain a lower bound for the log-likelihood

$$\ell(\theta) \geq \sum_i \mathrm{ELBO}(x^{(i)}; Q_i, \theta) \tag{11.11}$$

$$= \sum_i \sum_{z^{(i)}} Q_i(z^{(i)}) \log \frac{p(x^{(i)}, z^{(i)};\theta)}{Q_i(z^{(i)})}$$

For *any* set of distributions $Q_1, \ldots, Q_n$, the formula (11.11) gives a lower-bound on $\ell(\theta)$, and analogous to the argument around equation (11.8), the Q_i that attains equality satisfies

$$Q_i(z^{(i)}) = p(z^{(i)}|x^{(i)};\theta)$$

EM algorithm

Now, for this choice of the Q_i's, Equation (11.11) gives a lower-bound on the loglikelihood ℓ that we're trying to maximize. This is the E-step. In the M-step of the algorithm, we then maximize our formula in Equation (11.11) with respect to the parameters to obtain a new setting of the θ's. Repeatedly carrying out these two steps gives us the EM algorithm, which is as follows:

Repeat until convergence {

(E-step) For each i, set

$$Q_i(z^{(i)}) := p(z^{(i)}|x^{(i)};\theta).$$

(M-step) Set

$$\begin{aligned}\theta &:= \arg\max_\theta \sum_{i=1}^{n} \mathrm{ELBO}(x^{(i)}; Q_i, \theta) \\ &= \arg\max_\theta \sum_i \sum_{z^{(i)}} Q_i(z^{(i)}) \log \frac{p(x^{(i)}, z^{(i)};\theta)}{Q_i(z^{(i)})}. \qquad (11.12)\end{aligned}$$

}

ممکن است در max محلی گیر کند ...

EM algorithm

$$l(\theta^{(t)}) \leq l(\theta^{(t+1)})$$

همگرایی EM: به صورت یکنواخت در تکرارهای الگوریتم تابع $l(\theta)$ افزایشی است.

$$\left| \frac{l(\theta)^{(n+1)} - l(\theta)^{(n)}}{(l(\theta))^{(n)}} \right| < th$$ معیار توقف الگوریتم

$$l(\theta^{(t)}) = \sum_{i=1}^{m} ELBO(x^{(i)}, Q_i^{(t)}, \theta^{(t)})$$

$$\theta^{(t+1)} = \arg\max \sum_{i=1}^{m} ELBO(x^{(i)}, Q_i^{(t)}, \theta^{(t+1)})$$

$l(\theta^{(t)})$

$l(\theta^{(t+1)}) \overset{\text{Jensen}}{\geq} \sum_{i=1}^{m} ELBO(x^{(i)}, Q_i^{(t)}, \theta^{(t+1)}) \overset{ELBO \text{ maximum}}{\geq} \sum_{i=1}^{m} ELBO(x^{(i)}, Q_i^{(t)}, \theta^{(t)})$

Dimensionality Reduction

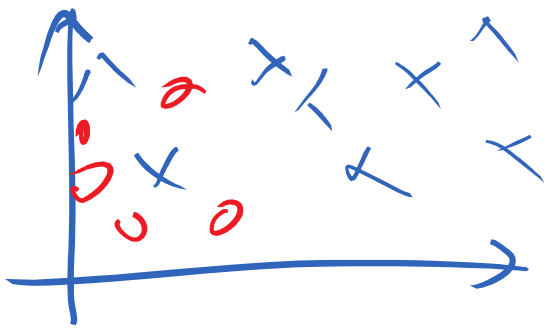
روش های کاهش بعد

۱- فشرده سازی

۲- Visualization

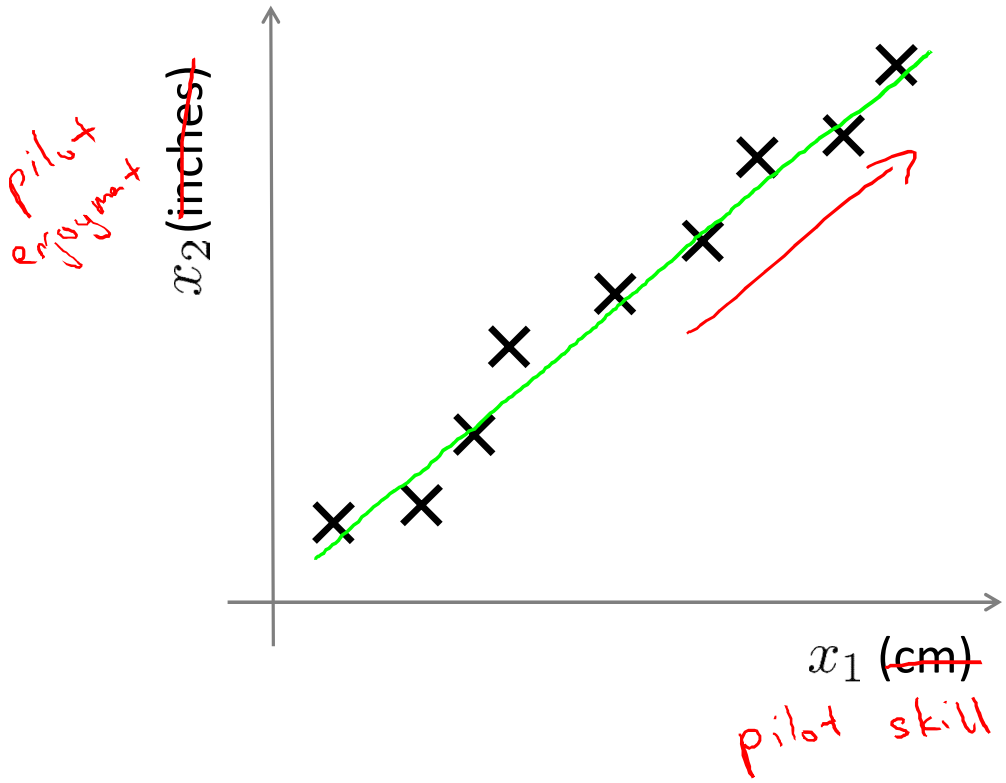
۳- سرطت کشیدن به الگوریتم یادگیری

300 → 2



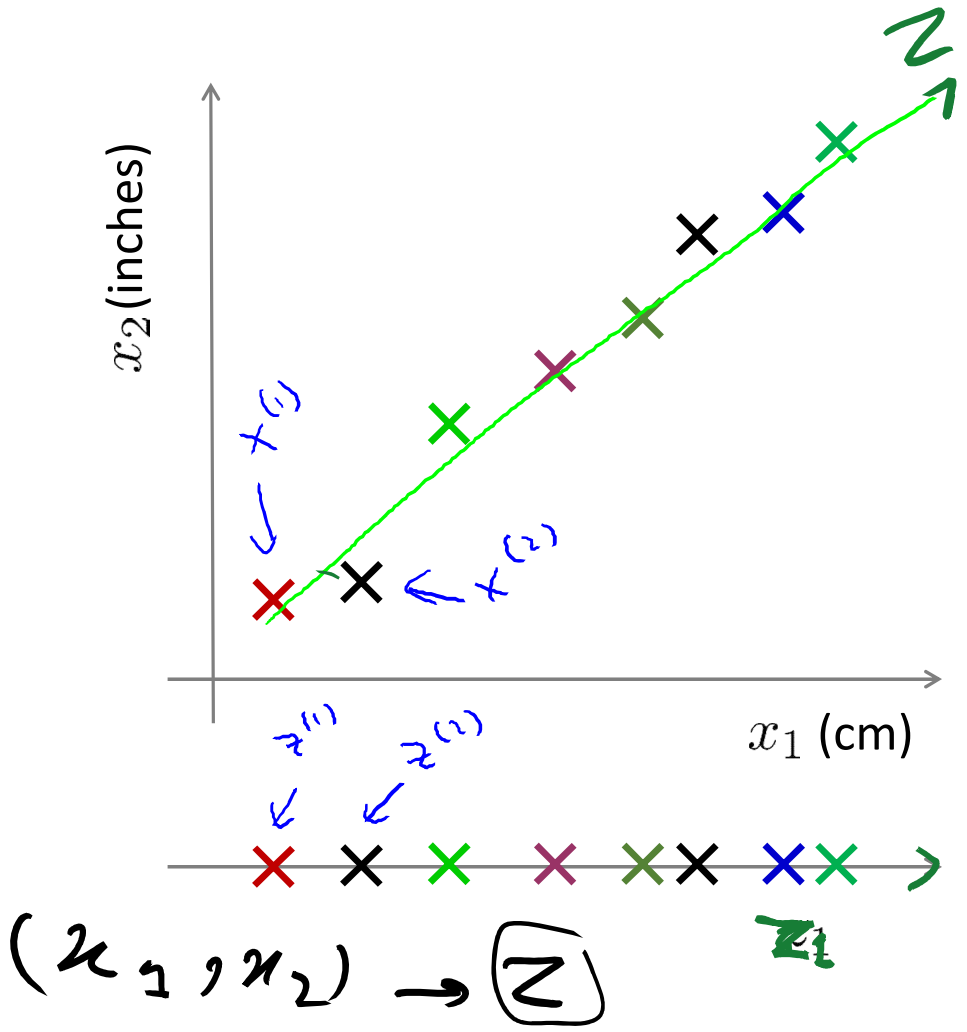
300 → 50

Data Compression



Reduce data from 2D to 1D

Data Compression



Reduce data from 2D to 1D

$$x^{(1)} \in \mathbb{R}^2 \rightarrow z^{(1)} \in \mathbb{R}$$

$$x^{(2)} \in \mathbb{R}^2 \rightarrow z^{(2)} \in \mathbb{R}$$

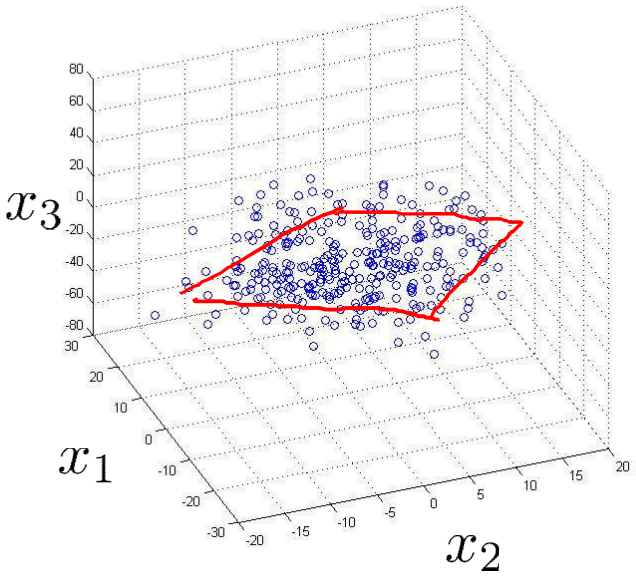
$$\vdots$$

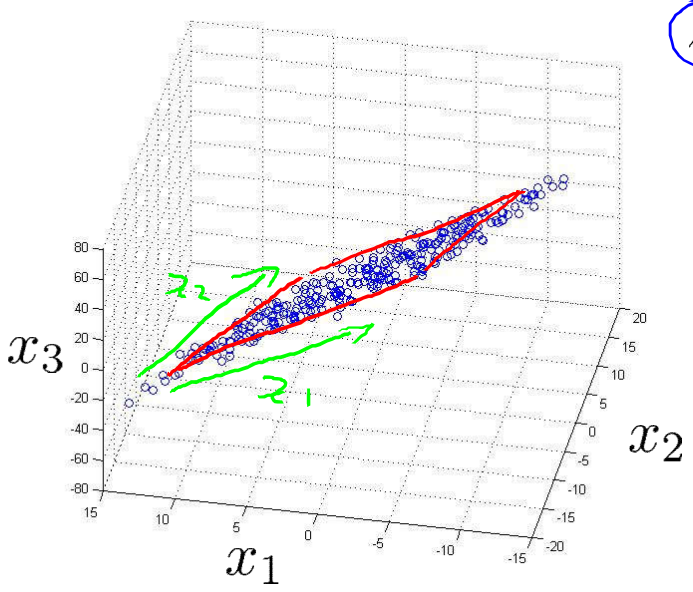
$$x^{(m)} \in \mathbb{R}^2 \rightarrow z^{(m)} \in \mathbb{R}$$

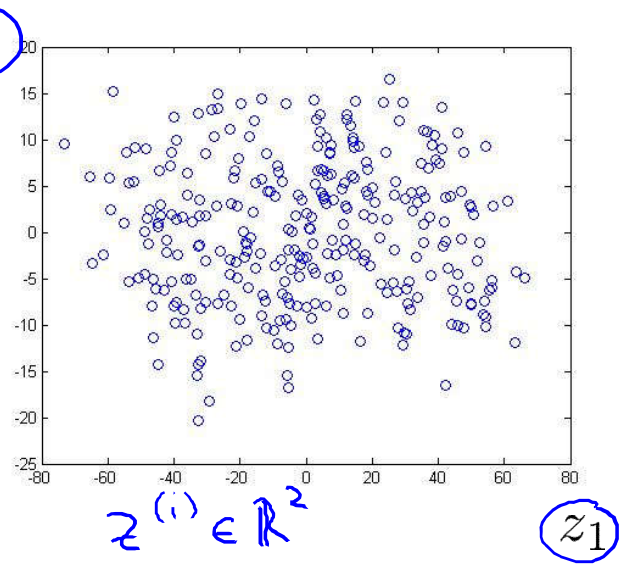
Data Compression

1000D → 100D

Reduce data from 3D to 2D







$x^{(i)} \in \mathbb{R}^3$

$z^{(i)} \in \mathbb{R}^2$

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \longrightarrow \begin{bmatrix} z_1 \\ z_2 \end{bmatrix}$$

$$z^{(i)} = \begin{bmatrix} z_1^{(i)} \\ z_2^{(i)} \end{bmatrix}$$

Data Visualization



$x^{(i)} \in \mathbb{R}^{50}$

Country	x_1 GDP (trillions of US$)	x_2 Per capita GDP (thousands of intl. $)	x_3 Human Develop-ment Index	x_4 Life expectancy	x_5 Poverty Index (Gini as percentage)	x_6 Mean household income (thousands of US$)	…
Canada	1.577	39.17	0.908	80.7	32.6	67.293	…
China	5.878	7.54	0.687	73	46.9	10.22	…
India	1.632	3.41	0.547	64.7	36.8	0.735	…
Russia	1.48	19.84	0.755	65.5	39.9	0.72	…
Singapore	0.223	56.69	0.866	80	42.5	67.1	…
USA	14.527	46.86	0.91	78.3	40.8	84.3	…
…	…	…	…	…	…	…	

[resources from en.wikipedia.org]

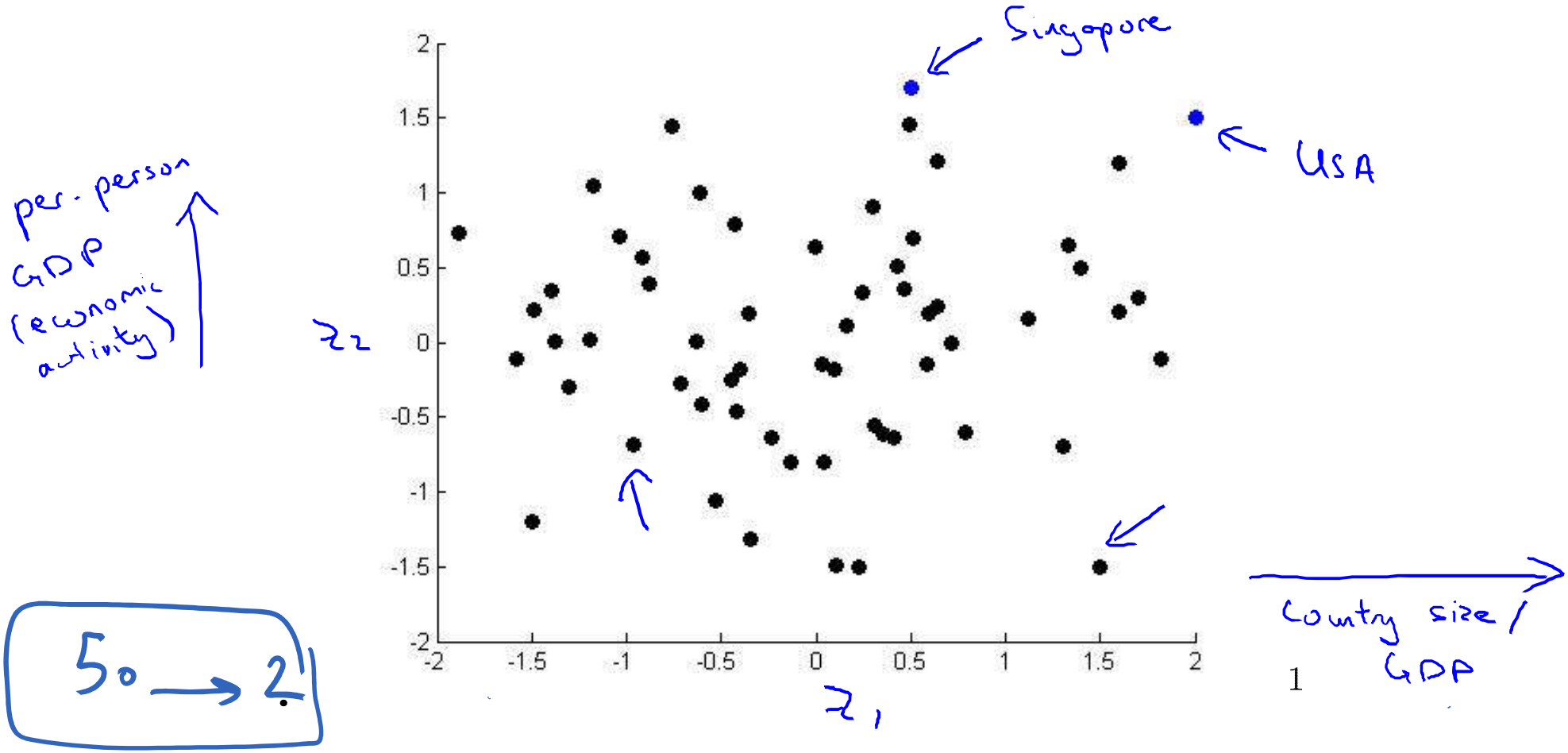
Data Visualization

Country	z_1	z_2
Canada	1.6	1.2
China	1.7	0.3
India	1.6	0.2
Russia	1.4	0.5
Singapore	0.5	1.7
USA	2	1.5
…	…	…

$z^{(i)} \in \mathbb{R}^2$

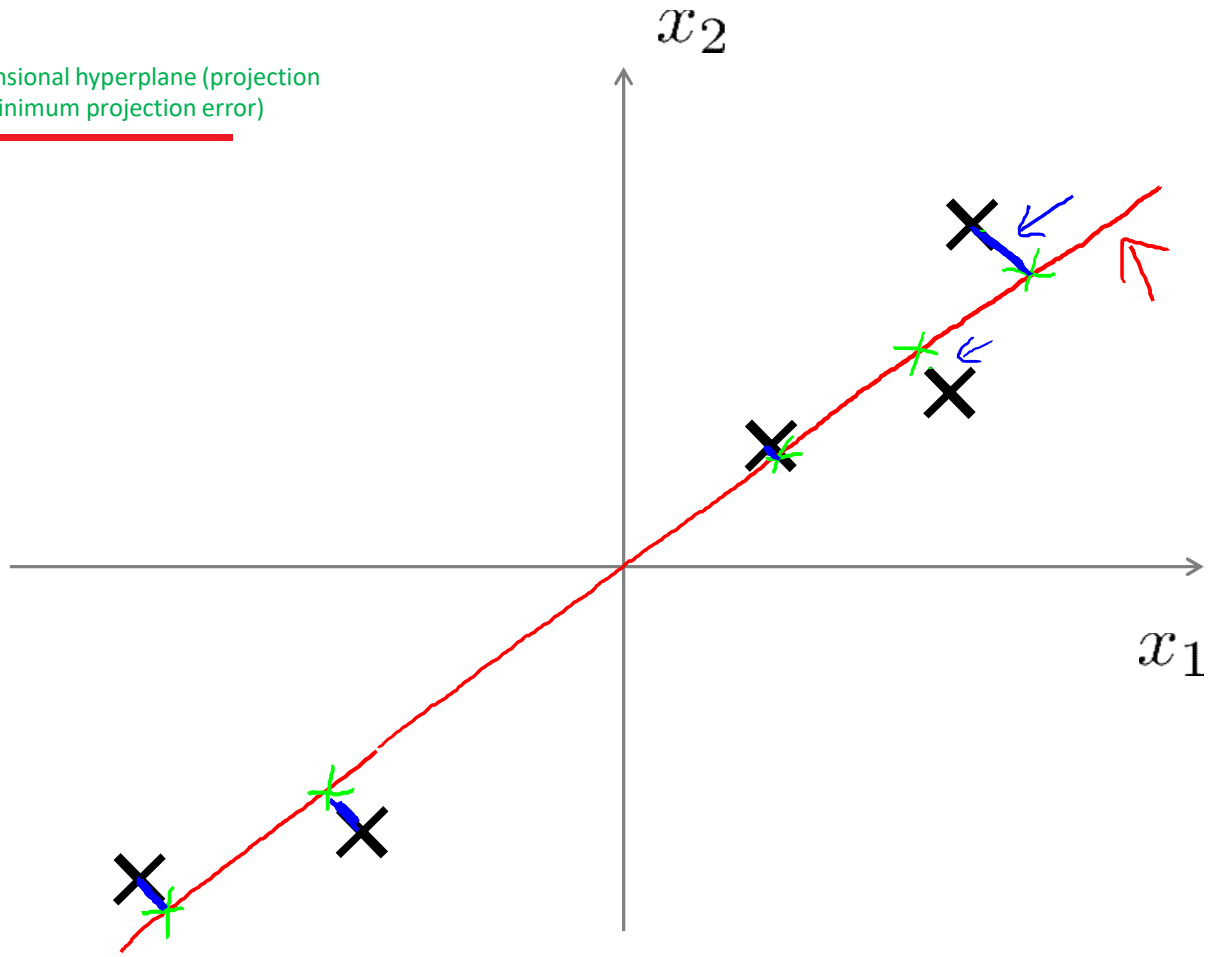
Reduce data from 50D to 2D

Data Visualization



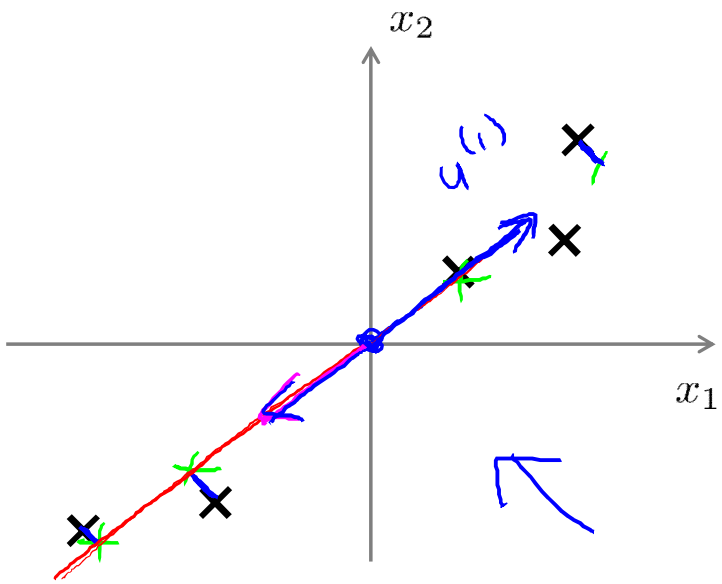
Principal Component Analysis (PCA) problem formulation

(PCA tries to find a low dimensional hyperplane (projection hyperplane) which has the minimum projection error)

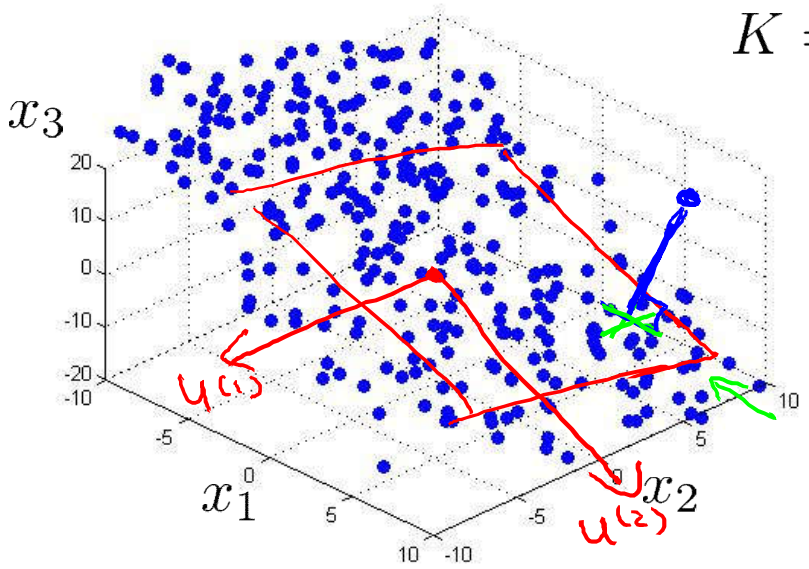


$x \in \mathbb{R}^2$

Principal Component Analysis (PCA) problem formulation



$3D \rightarrow 2D$
$K = 2$

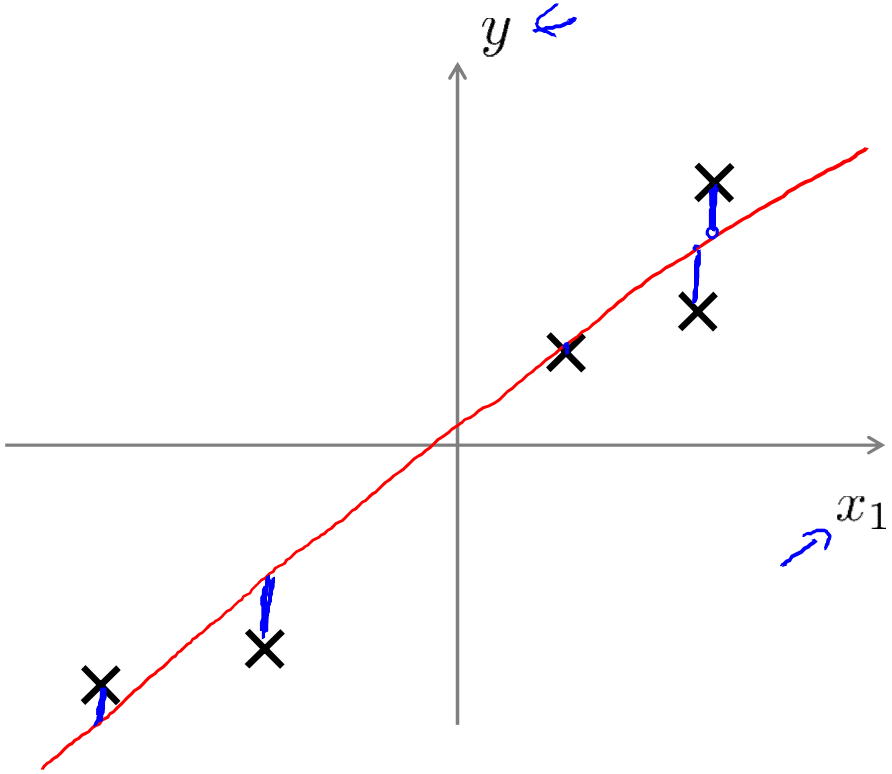


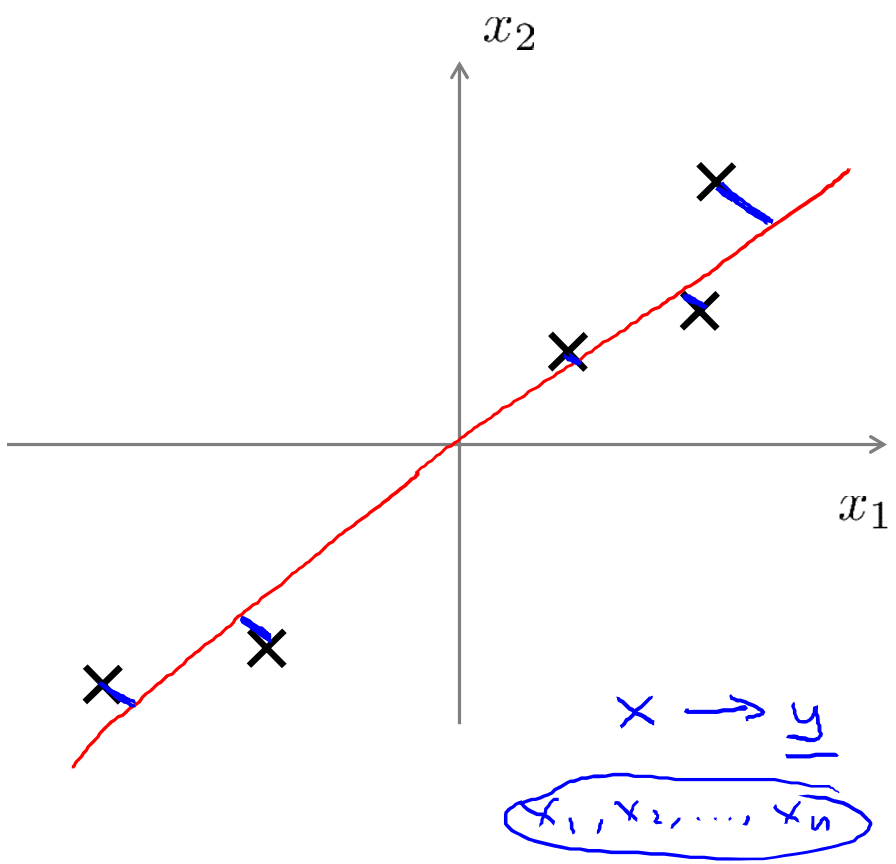
Reduce from 2-dimension to 1-dimension: Find a direction (a vector $u^{(1)} \in \mathbb{R}^n$) onto which to project the data so as to minimize the projection error.

Reduce from n-dimension to k-dimension: Find $k$vectors $u^{(1)}, u^{(2)}, \ldots, u^{(k)}$ onto which to project the data, so as to minimize the projection error.

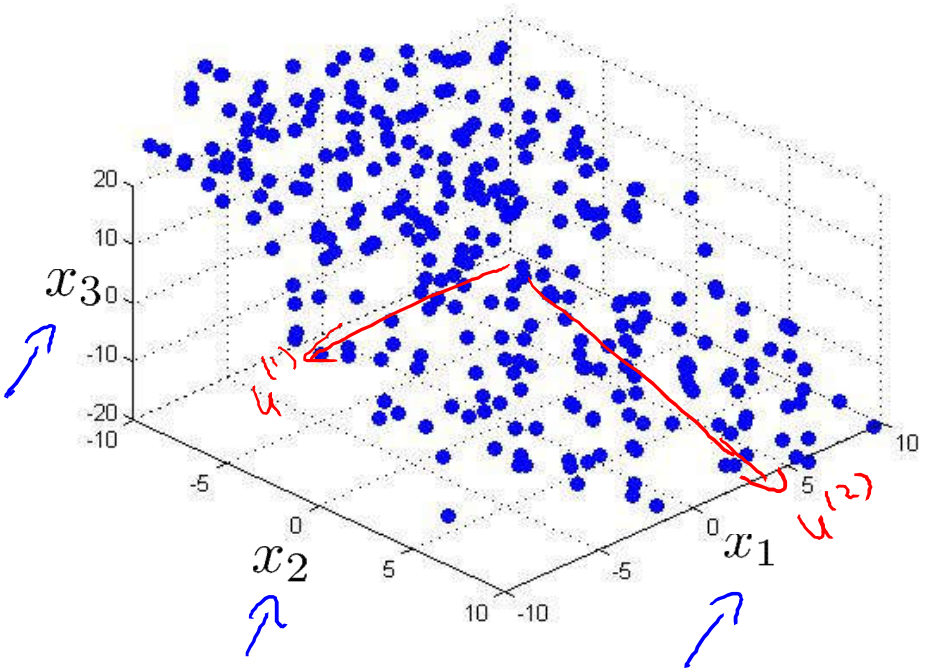
PCA is not linear regression

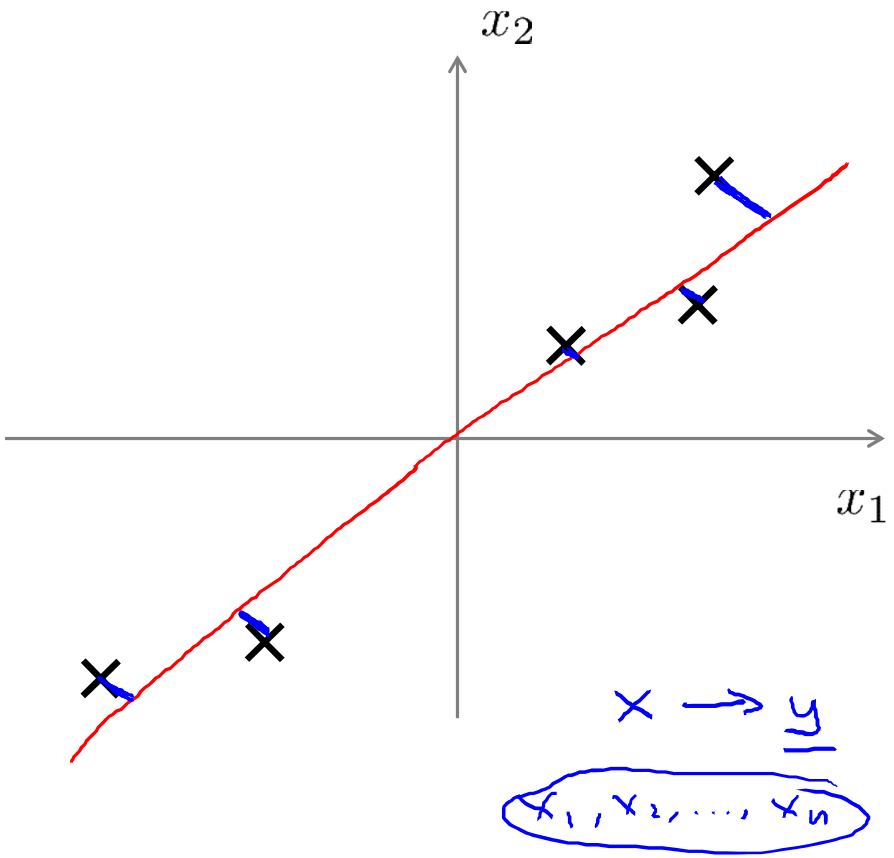
(PCA tries to minimize (squared) projection error not MSE)





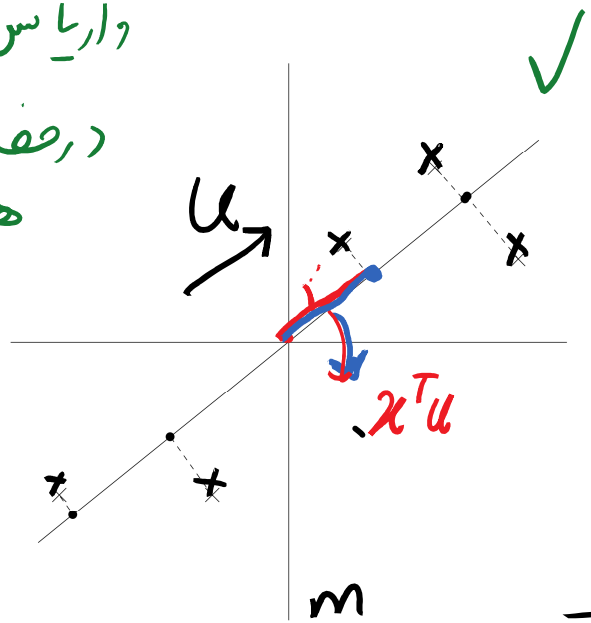
PCA is not linear regression



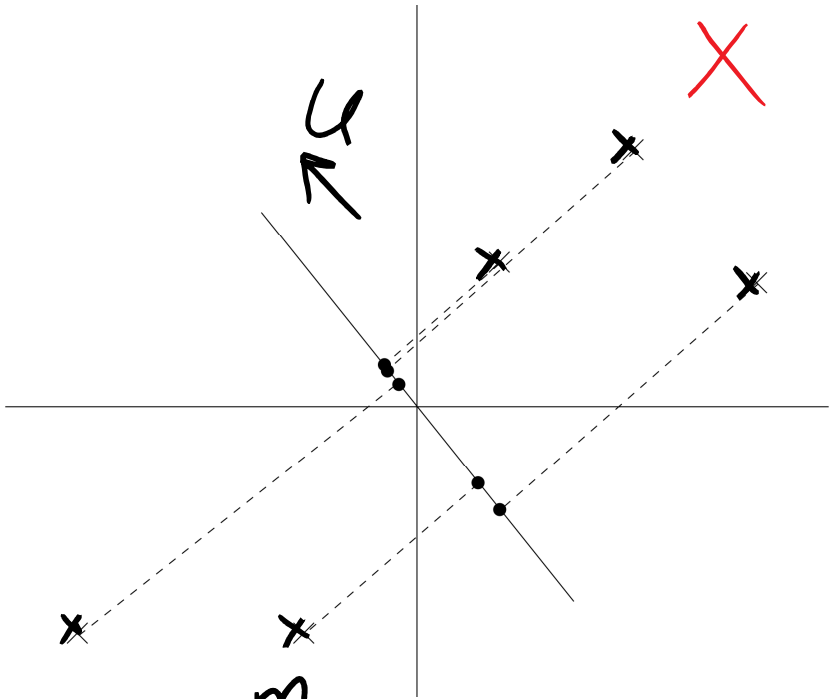


Principal Component Analysis (PCA)

واریانس داده‌ها در فضای جدید هم حفظ شود.



✓



✗

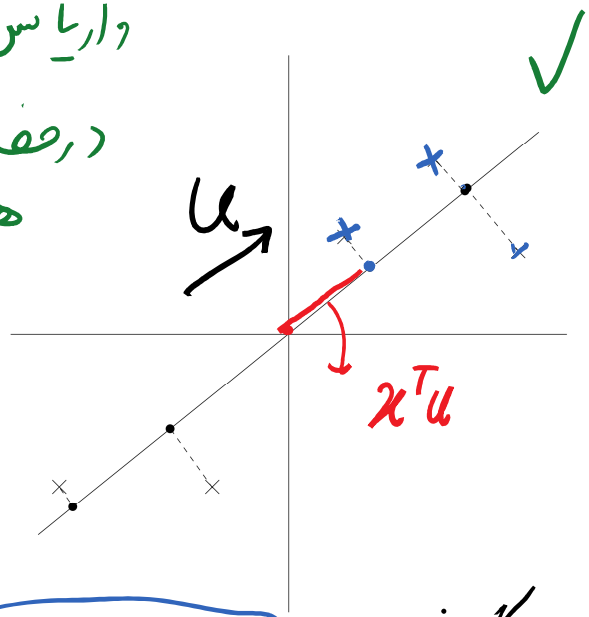
با فرض $\|u\| = 1$

$$\max_u \frac{1}{m} \sum_{i=1}^{m} (x^{(i)T} u)^2 = \max \frac{1}{m} \sum_{i=1}^{m} u^T x^{(i)} x^{(i)T} u$$

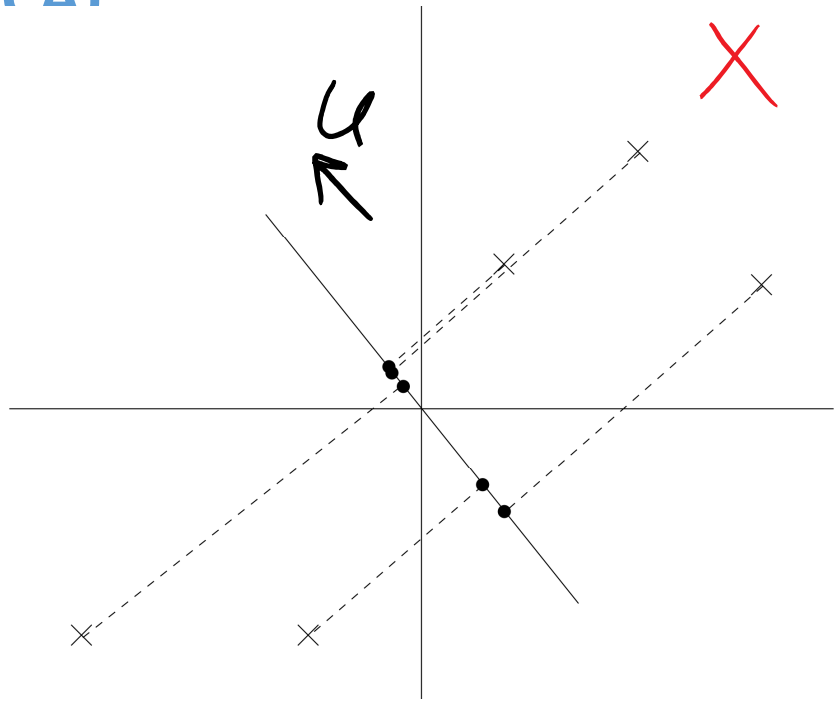
$$\max_u u^T \left(\frac{1}{m} \sum_{i=1}^{m} x^{(i)} x^{(i)T} \right) u \rightarrow \Sigma \quad (\text{با فرض صفر بودن } \mu^{(i)})$$

Principal Component Analysis (PCA)

واریانس داده‌ها در فضای جدید هم حفظ شود.



✓



X

$$\max_{u} \; u^T \Sigma u \quad \text{s.t.} \quad u^T u = 1$$

بازنویسی لاگرانژ

$\Sigma u = \lambda u$

max

u: بردار ویژه اساسی Σ

Principal Component Analysis (PCA)

برای داشتن بعد از n به k: یافتن $u_1, u_2, \dots, u_k$

u_1 تا u_k ← k بردار ویژه متناظر با k بزرگترین مقدار ویژه

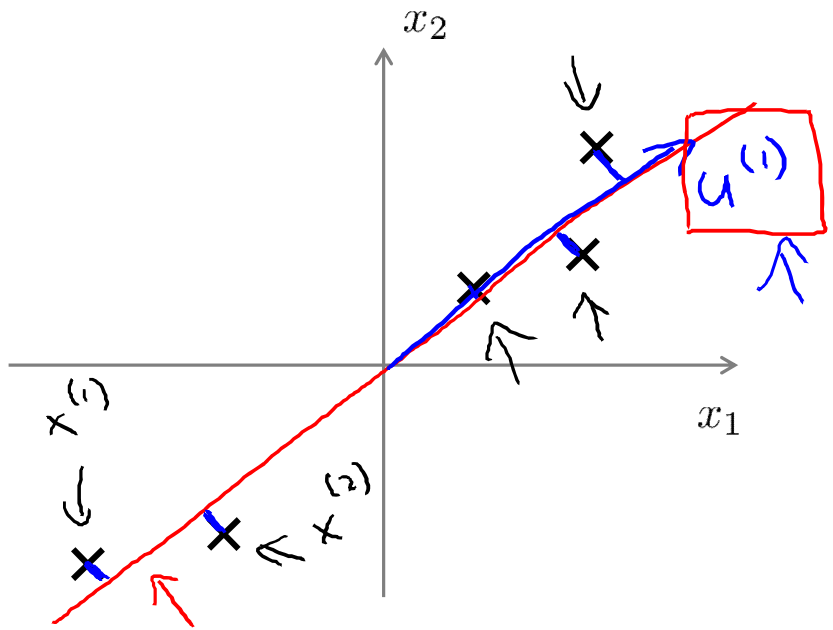
این بردارها جهت k مؤلفه‌هایی را تعیین می‌کنند

principal components

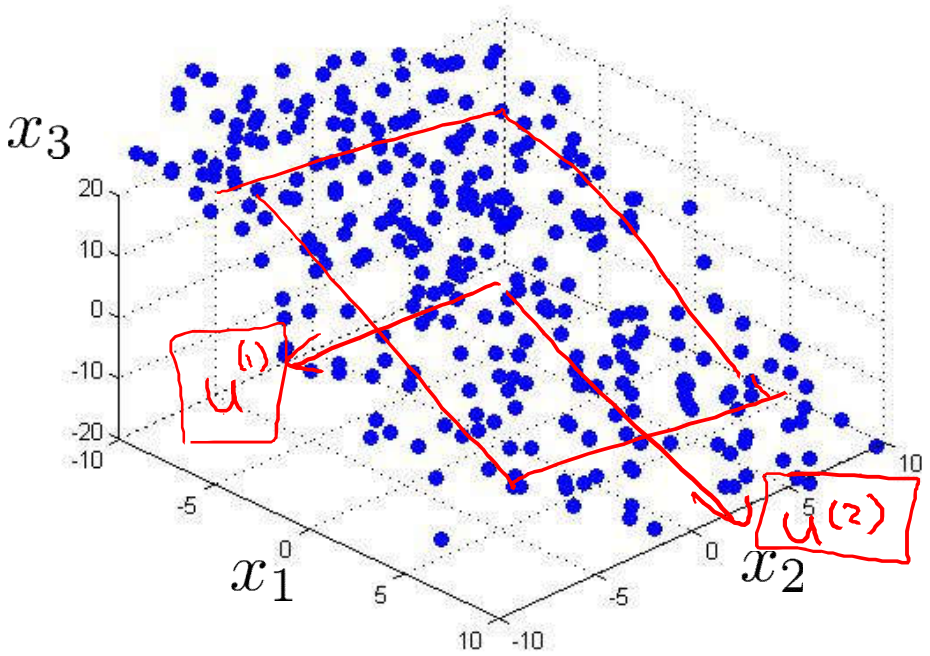
* حداکثر کردن واریانس مدل یا حداقل کردن خطای مابین نقاط و تصویرهای آنها

Projection error

Principal Component Analysis (PCA) algorithm



Reduce data from 2D to 1D



Reduce data from 3D to 2D

$x^{(1)} \in \mathbb{R}^2 \rightarrow z^{(1)} \in \mathbb{R}$

z_1

$x^{(1)} \in \mathbb{R}^3 \rightarrow z^{(1)} \in \mathbb{R}^2$

$z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix}$

Symbol *sigma* representing covariance matrix

Principal Component Analysis (PCA) algorithm

Reduce data from n-dimensions to k-dimensions

Compute "covariance matrix":

$$\Sigma = \frac{1}{m} \sum_{i=1}^{m} (x^{(i)})(x^{(i)})^T$$

$n \times 1$ $\quad$ $1 \times n$ $\quad$ $n \times n$

Compute "eigenvectors" of matrix Σ:

```
[U,S,V] = svd(Sigma);
```

→ Singular value decomposition

eig (Sigma)

$n \times n$ matrix.

$$U = \begin{bmatrix} | & | & | & & | \\ u^{(1)} & u^{(2)} & u^{(3)} & \cdots & u^{(n)} \\ | & | & | & & | \end{bmatrix}$$

k

$U \in \mathbb{R}^{n \times n}$

$u^{(1)}, \ldots, u^{(k)}$

Principal Component Analysis (PCA) algorithm

From `[U,S,V] = svd(Sigma)`:

$$\Rightarrow U = \begin{bmatrix} | & | & & | \\ u^{(1)} & u^{(2)} & \cdots & u^{(n)} \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{n \times n}$$

k

$x \in \mathbb{R}^n \longrightarrow z \in \mathbb{R}^k$

$$z^{(i)} = \begin{bmatrix} | & | & & | \\ u^{(1)} & u^{(2)} & \cdots & u^{(k)} \\ | & | & & | \end{bmatrix}^T x^{(i)} = \begin{bmatrix} — (u^{(1)})^T — \\ \vdots \\ — (u^{(k)})^T — \end{bmatrix} x^{(i)}$$

$z \in \mathbb{R}^k$ $\quad n \times k$ $\quad k \times n$ $\quad n \times 1$ $\quad k \times 1$

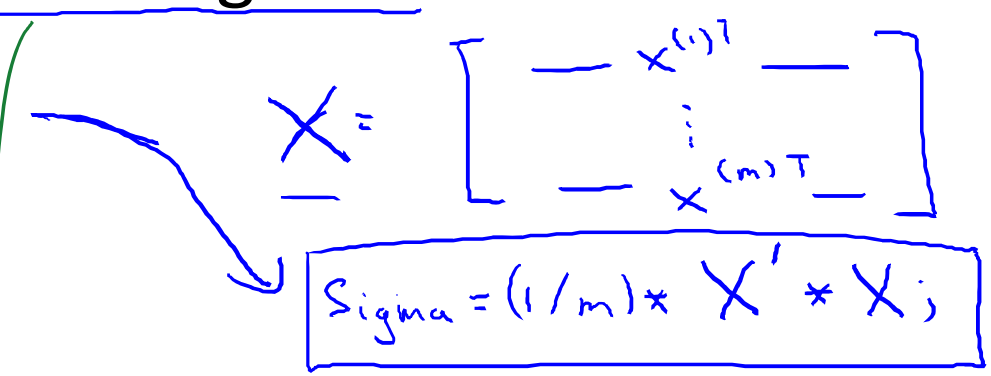
U_{reduce}

Principal Component Analysis (PCA) algorithm summary

- After mean normalization (ensure every feature has zero mean) and optionally feature scaling:

Sigma = $\frac{1}{m}\sum_{i=1}^{m}(x^{(i)})(x^{(i)})^T$

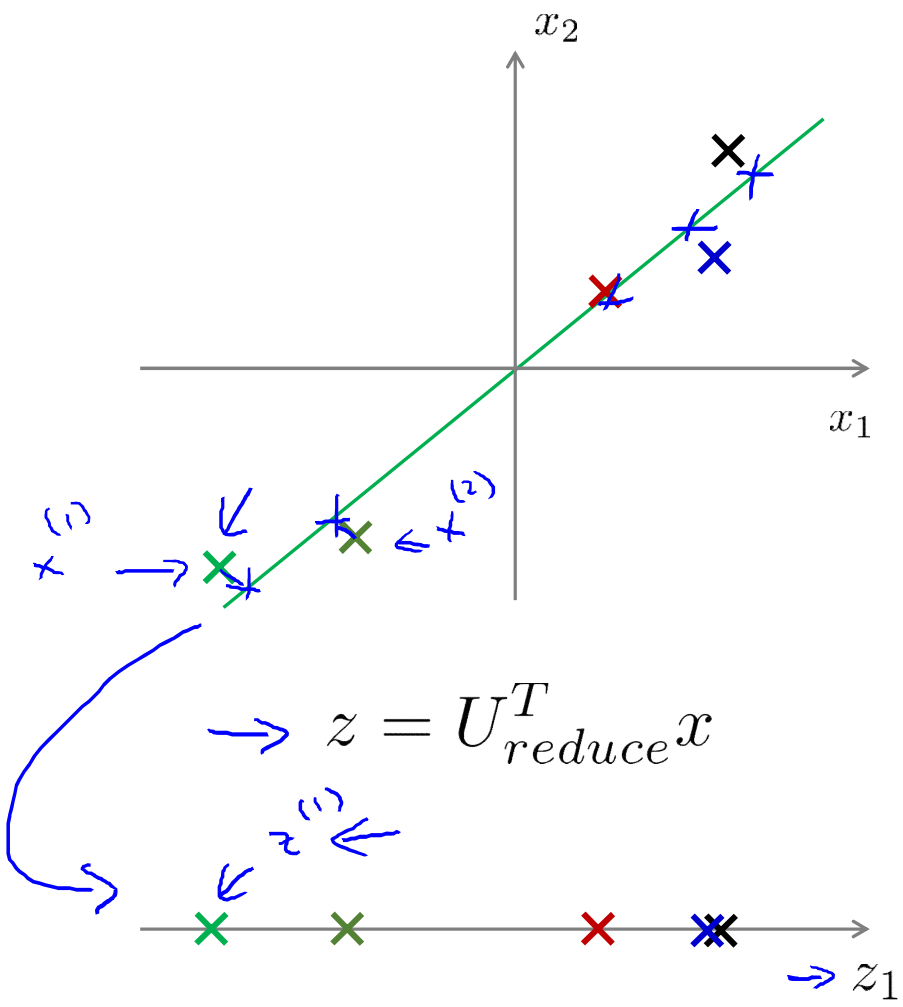
- `[U,S,V] = svd(Sigma);`
- `Ureduce = U(:,1:k);`
- `z = Ureduce'*x;`

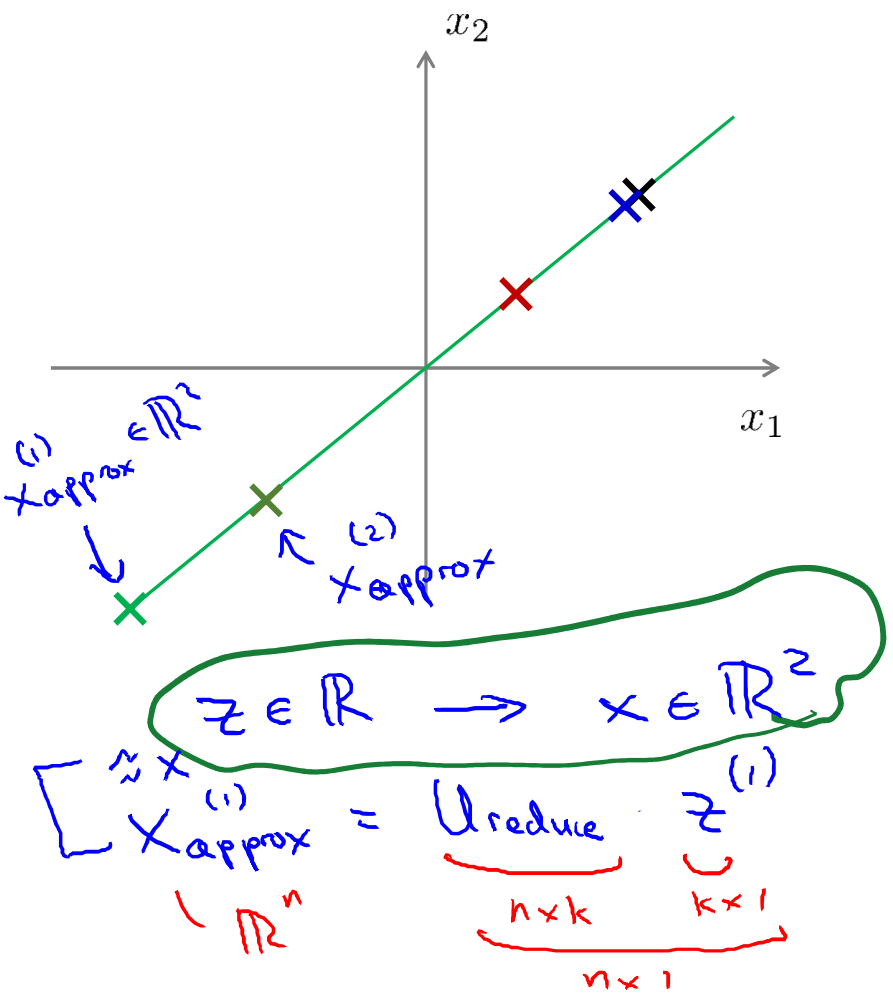


$x_j^{(i)} = \frac{x_j^{(i)} - \mu_j}{std_j}$ $\quad j = 1 \ldots n$

$\mu_j = \frac{1}{m}\sum_{i=1}^{m} x_j^{(i)}$

Reconstruction from compressed representation





Choosing k (number of principal components)

Average squared projection error: $\frac{1}{m}\sum_{i=1}^{m} \|x^{(i)} - x_{approx}^{(i)}\|^2$

Total variation in the data: $\frac{1}{m}\sum_{i=1}^{m} \|x^{(i)}\|^2$

Typically, choose k to be smallest value so that

$$\frac{J}{TV} = \frac{\frac{1}{m}\sum_{i=1}^{m} \|x^{(i)} - x_{approx}^{(i)}\|^2}{\frac{1}{m}\sum_{i=1}^{m} \|x^{(i)}\|^2} \leq 0.01 \quad (1\%)$$

0.05 (5%)

0.10 (10%)

→ "99% of variance is retained"

95% 90%

Choosing k (number of principal components)

Algorithm:

Try PCA with $k = 1$ k=2 K=3 ...

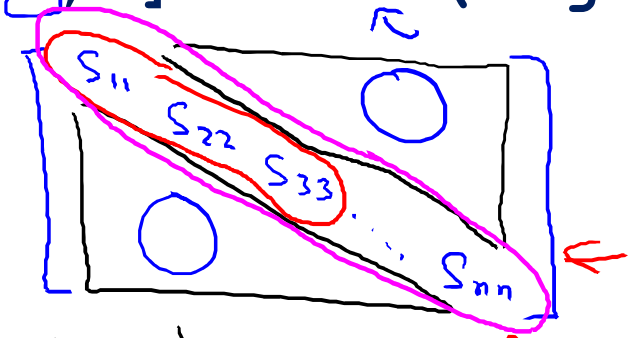
Compute $U_{reduce}, z^{(1)}, z^{(2)}, \ldots, z^{(m)}, x^{(1)}_{approx}, \ldots, x^{(m)}_{approx}$

Check if

$$\frac{\frac{1}{m}\sum_{i=1}^{m} \|x^{(i)} - x^{(i)}_{approx}\|^2}{\frac{1}{m}\sum_{i=1}^{m} \|x^{(i)}\|^2} \leq 0.01?$$

→ `[U,S,V] = svd(Sigma)`

→ S =



k=3

For given k

$$1 - \frac{\sum_{i=1}^{k} S_{ii}}{\sum_{i=1}^{n} S_{ii}} \leq 0.01$$

$$\rightarrow \frac{\sum_{i=1}^{k} S_{ii}}{\sum_{i=1}^{n} S_{ii}} \geq 0.99$$

Choosing k (number of principal components)

`[U,S,V] = svd(Sigma)`

Pick smallest value of k for which

$$\frac{\sum_{i=1}^{k} S_{ii}}{\sum_{i=1}^{n} S_{ii}} \geq 0.99$$

(99% of variance retained)

$k = 100$

Supervised learning speedup

$(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})$

Extract inputs:

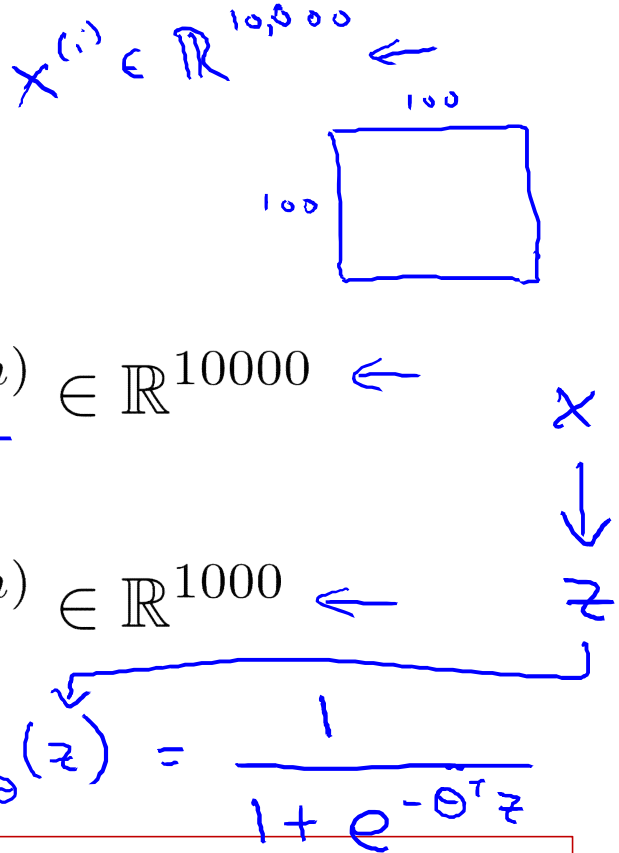
Unlabeled dataset: $x^{(1)}, x^{(2)}, \ldots, x^{(m)} \in \mathbb{R}^{10000}$

$\downarrow PCA$

$z^{(1)}, z^{(2)}, \ldots, z^{(m)} \in \mathbb{R}^{1000}$

New training set:

$(z^{(1)}, y^{(1)}), (z^{(2)}, y^{(2)}), \ldots, (z^{(m)}, y^{(m)})$



Note: Mapping $x^{(i)} \rightarrow z^{(i)}$ should be defined by **running PCA only on the training set**. This mapping can be applied as well to the examples $x_{cv}^{(i)}$ and $x_{test}^{(i)}$ in the cross validation and test sets.

Application of PCA

$x^{(i)} \rightarrow \phi(x^{(i)})$

$x^{(i)} x^{(j)T} \rightarrow \text{Kernel}(x^{(i)}, x^{(j)})$

PCA → Kernel PCA

- Compression
  - Reduce memory/disk needed to store data
  - Speed up learning algorithm ←

Choose k by % of variance retained

- Visualization

$k = 2$ or $k = 3$

- noise reduction

فیلتر کردن نویز ← $x^{(i)}$ 100×100 پیکسل $z^{(i)}$ ۱۰۰۰ ویژگی

Bad use of PCA: To prevent overfitting

- Use $z^{(i)}$ instead of $x^{(i)}$ to reduce the number of features to $k < n$. (1000 < 10000)
  Thus, fewer features, less likely to overfit.

Bad!

This might work OK, but isn't a good way to address overfitting. Use regularization instead.

$$\min_\theta \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m}\sum_{j=1}^{n}\theta_j^2$$

PCA is sometimes used where it shouldn't be

Design of ML system:

- Get training set $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})\}$
- ~~Run PCA to reduce $x^{(i)}$ in dimension to get $z^{(i)}$~~
- Train logistic regression on $\{(z^{(1)}, y^{(1)}), \ldots, (z^{(m)}, y^{(m)})\}$
- Test on test set: Map $x^{(i)}_{test}$ to $z^{(i)}_{test}$. Run $h_\theta(z)$ on $\{(z^{(1)}_{test}, y^{(1)}_{test}), \ldots, (z^{(m)}_{test}, y^{(m)}_{test})\}$

How about doing the whole thing without using PCA?

Before implementing PCA, first try running whatever you want to do with the original/raw data $x^{(i)}$. Only if that doesn't do what you want, then implement PCA and consider using $z^{(i)}$.

یادآوری یافتن مقادیر ویژه و بردارهای ویژه ماتریس $A_{n\times n}$.

$$A\bar{v} = \lambda \bar{v} \qquad (A-\lambda I)\bar{v} = 0 \qquad \det(A-\lambda I) = 0$$

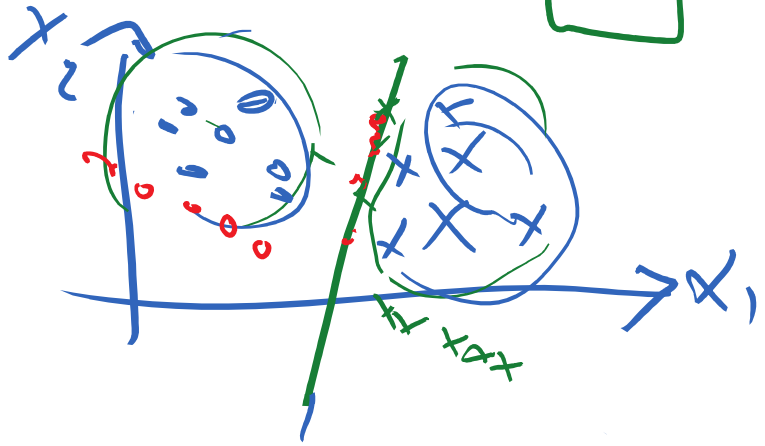
$$\Rightarrow \quad \lambda_1 \quad \lambda_2 \quad \cdots \quad \lambda_n$$
$$\downarrow \quad \downarrow \qquad \downarrow$$
$$\bar{v}_1 \quad \bar{v}_2 \qquad \bar{v}_n$$

$$A\bar{v}_1 = \lambda_1 v_1 \longrightarrow \bar{v}_1 \quad \ldots$$

Linear Discriminant Analysis

Linear Discriminant Analysis

- LDA is a supervised whereas PCA is unsupervised – PCA ignores class labels.
- Whereas PCA seeks directions that are efficient for representation, discriminant analysis seeks directions that are efficient for discrimination
- Maximum size of reduction is C-1



Linear Discriminant Analysis

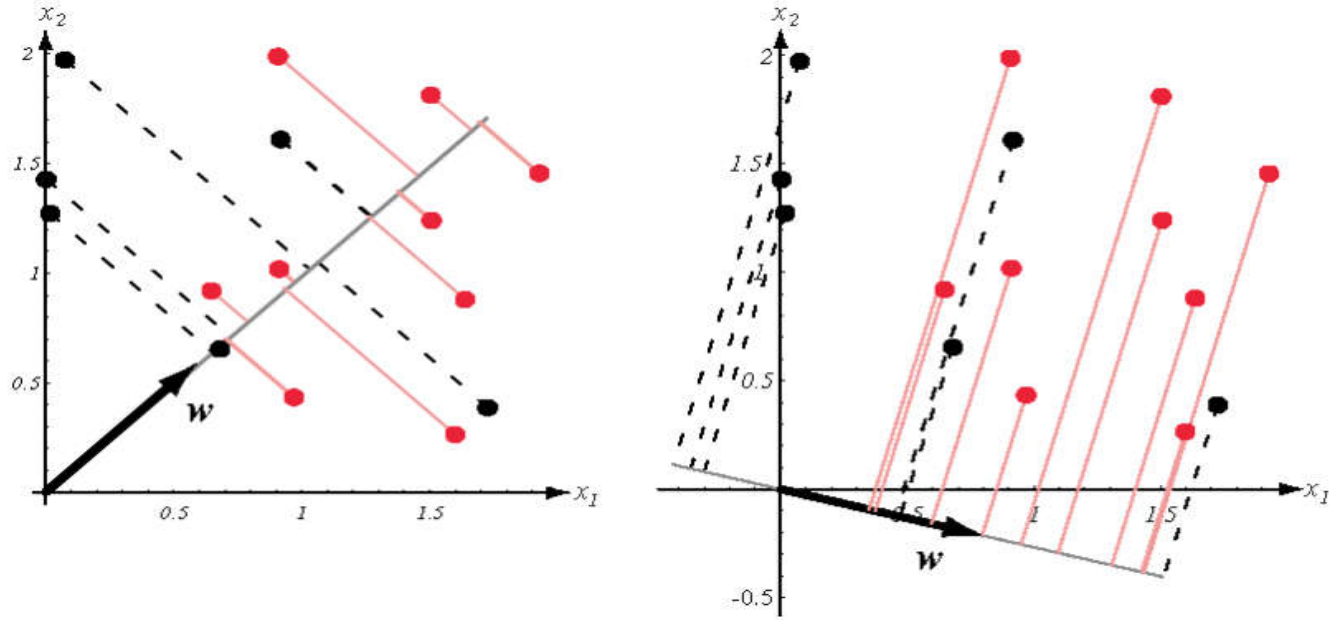
Notation

$y^i \epsilon \{1,2, \dots, C\}$

$i = 1, \dots, C$

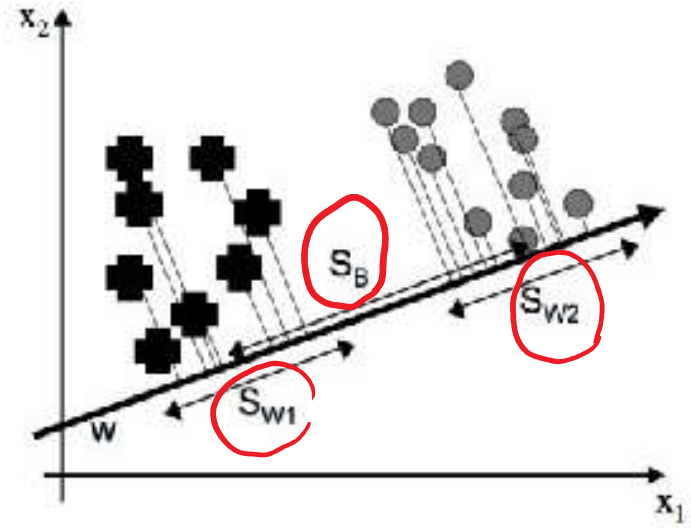
N_i: Number of instance in class i

Linear Discriminant Analysis

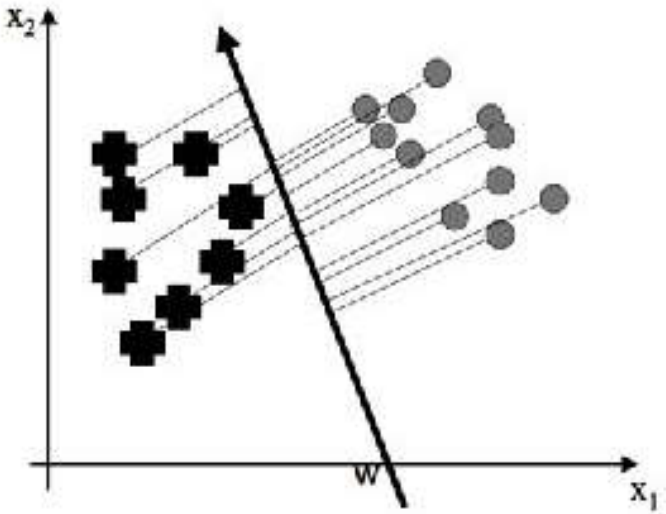


Projection of the same set of samples onto two different lines in the directions marked as **w**.
The figure on the right shows greater separation between the red and black projected points.

Linear Discriminant Analysis



Good projection



Bad projection

Linear Discriminant Analysis

- Given $x_1, \ldots, x_m \in \mathbb{R}^n$ divided into two subsets $\mathcal{D}_1$ and $\mathcal{D}_2$ corresponding to the classes c_1 and c_2, respectively, the goal is to find a projection onto a line defined as

$$y = \mathbf{w}^T \mathbf{x}$$

$x \rightarrow y$

where the points corresponding to $\mathcal{D}_1$ and $\mathcal{D}_2$ are well separated.

$$W_{n*C-1}$$

$$y \in R^{c-1}$$

$$C = 2 \longrightarrow W_{n*1} = \begin{bmatrix} w_1 \\ \vdots \\ w_n \end{bmatrix}$$

$$y_{1\times 1} = w^T_{1\times n} x_{n\times 1}$$

Linear Discriminant Analysis

LDA for C = 2

- Average distance between classes must be maximum

$$\mu_i = \frac{1}{N_i} \sum_{x \epsilon C_i} x \qquad \hat{\mu}_i = \frac{1}{N_i} \sum_{x \epsilon C_i} y = W^T \mu_i$$

$$J(\omega) = |\tilde{\mu}_1 - \tilde{\mu}_2| = \left|W^T \mu_1 - W^T \mu_2\right| = \left|W^T (\mu_1 - \mu_2)\right|$$

Linear Discriminant Analysis

LDA for C = 2

- Std for each class must be minimized

$$\tilde{S}_i^2 = \sum_{y \in c_i} (y - \tilde{\mu}_i)^2$$

$$\tilde{S}_1^2 + \tilde{S}_2^2$$

Fisher: $$J(W) = \frac{|\tilde{\mu}_1 - \tilde{\mu}_2|^2}{\tilde{s}_1^2 + \tilde{S}_2^2}$$

between class scatter

within class scatter

$$S_i = \sum_{x \in c_i} (x - \mu_i)(x - \mu_i)^T$$

$$S_W = S_1 + S_2$$

$$\tilde{S}_i^2 = \sum_{y \in c_i} (y - \tilde{\mu}_i)^2 = \sum_{x \in c_i} W(x - \mu_i)(x - \mu_i)W = w^T S_i w$$

$$\tilde{S}_1^2 + \tilde{S}_2^2 = W^T(S_1 + S_2)w = W^T S_W W = \tilde{S}_W$$

within class scatter

Linear Discriminant Analysis

LDA for C = 2

$$S_i = \sum_{x \in c_i} (x - \mu_i)(x - \mu_i)^T \qquad S_W = S_1 + S_2$$

$$\tilde{S}_i^2 = \sum_{y \in c_i} (y - \tilde{\mu}_i)^2 = \sum_{x \in c_i} W(x_\mu_i)(x_\mu_i)W = \omega^T S_i \omega$$

$$\tilde{S}_1^2 + \tilde{S}_2^2 = W^T(S_1 + S_2)w = W^T S_W W = \tilde{S}_W$$

within class scatter

Linear Discriminant Analysis

LDA for C = 2

$$(\tilde{\mu}_1 - \tilde{\mu}_2)^2 = W^T S_B W = \tilde{S}_B$$

$$S_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T$$

→ between class scatter

Linear Discriminant Analysis

LDA for C = 2

the criterion function becomes

fisher criterion $\max J(w) = \frac{w^T S_{BW}}{w^T S_w w}$

$S_B w - J(w) S_w w = 0$ → $S_w^{-1} S_B w = J(w) w$

Eigenvector

and the optimal **w** can be computed as

$$\mathbf{w}^*_{n_x 1} = \mathbf{S}_\mathbf{W}^{-1}(\mu_1 - \mu_2)$$

Note that, $\mathbf{S_W}$ is symmetric and positive semidefinite, and it is usually nonsingular if m> n. $\mathbf{S_B}$ is also symmetric and positive semidefinite, but its rank is at most 1.

مثال:

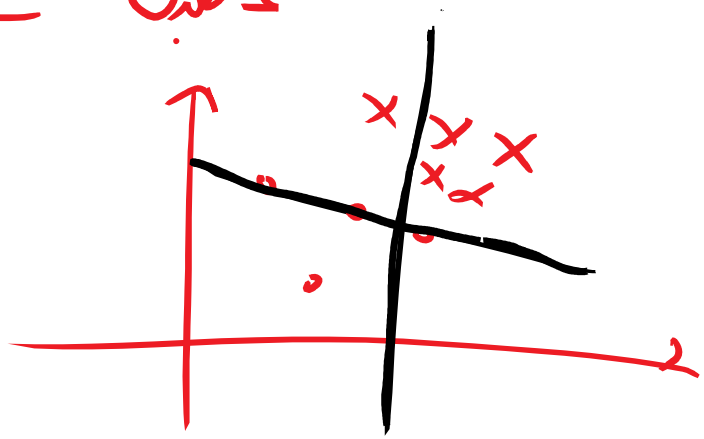
کلاس ①

$$X_1 = \begin{bmatrix} 4,2 \\ 2,4 \\ 2,3 \\ 3,6 \\ 4,4 \end{bmatrix}$$

کلاس ②

$$X_2 = \begin{bmatrix} 9,10 \\ 6,8 \\ 9,5 \\ 8,7 \\ 10,8 \end{bmatrix}$$

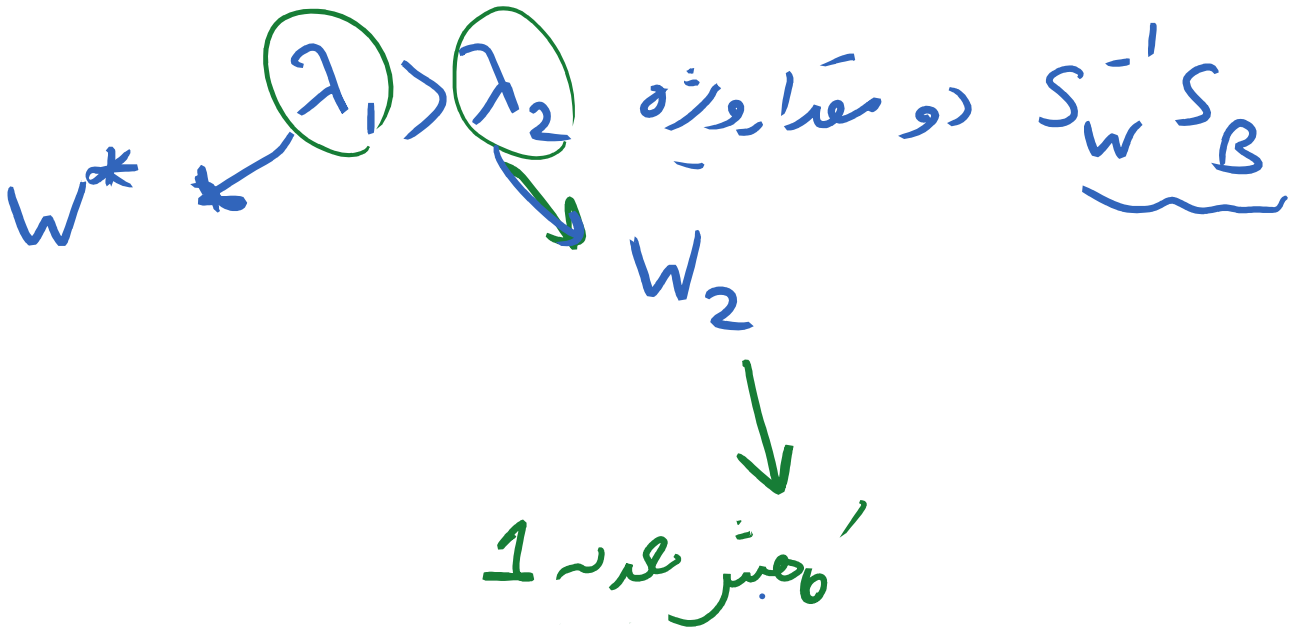
LDA → بعد 1



$\underbrace{\mu_1}_{2\times 1}, \underbrace{\mu_2}_{2\times 1} \rightarrow \underbrace{S_1}, \underbrace{S_2} \rightarrow S_W, S_B$

W^* بردار ویژه متناظر با بزرگترین مقدار ویژه $S_W^{-1} S_B$

$$W^* = S_W^{-1}(\mu_1 - \mu_2)$$


$\lambda_1 > \lambda_2$
دو مقدار ویژه
$S_W^{-1} S_B$
W^*
W_2
حبس عدد 1

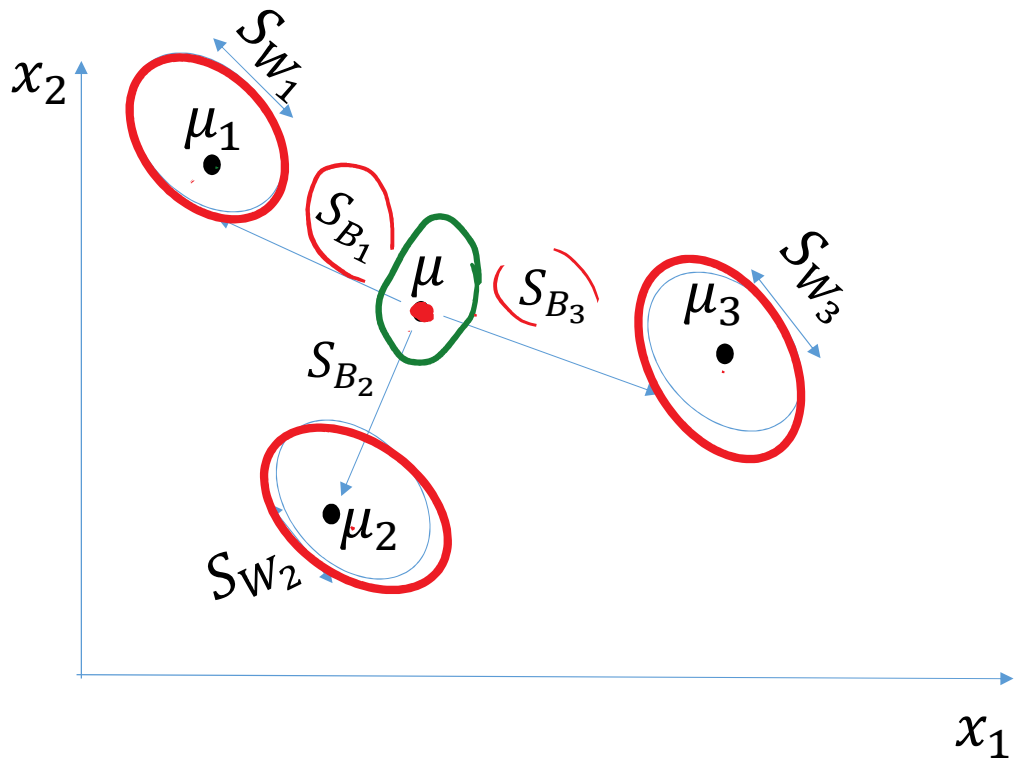
Linear Discriminant Analysis

LDA for C > 2

ماتریس W

$$y_{C-1 \times 1} = W^T_{C-1 \times n} \, x_{n \times 1}$$

$W_{n \times C-1}$



Linear Discriminant Analysis

LDA for C > 2

Vectors

$y_1 , , , y_{c-1} = ?$ → w_i

$$\mathrm{w} = \left[w_1 | w_2 | \cdots w_{c_{-1}} \right]_{n \times c-1}$$

$$x_{n \times 1}$$

$$y_i = w_i^T x \rightarrow y_{c-1 \times 1} = w^\top x_{n \times 1}$$

$(c-1) \times n$

Linear Discriminant Analysis

LDA for C > 2

$$S_w = \sum_{i=1}^{c} S_i = \sum_{i=1}^{c} \sum_{x \in c_i} (x - \mu_i)(x - \mu_i)^{\mathsf{T}}$$

$$\mu_i = \frac{1}{N_i} \sum_{x \in c_i} x$$

$$S_B = \sum_{i=1}^{c} N_i (\mu_i - \mu)(\mu_i - \mu)^T$$

$$\mu = \frac{1}{N} \sum_{i=1}^{c} x = \frac{1}{N} \sum_{i=1}^{C} N_i \mu_i$$

Linear Discriminant Analysis

LDA for C > 2

$$\tilde{\mu}_i = \frac{1}{Ni} \sum_{y \in C_i} y$$

$$\tilde{\mu} = \frac{1}{N} \sum_{\forall y}^{N} y$$

$$\tilde{S}_w = \sum_{i=1}^{c} \tilde{S}_i = \sum_{i=1}^{c} \sum_{y \epsilon C_i} (y - \tilde{\mu}_i)(y - \tilde{\mu}_i)^{\top}$$

$$\tilde{S}_{\mathrm{B}} = \sum_{i=1}^{c} N_i (\tilde{\mu}_i - \tilde{\mu})(\tilde{\mu}_i - \tilde{\mu})^T$$

$$\begin{cases} \tilde{S}_w = \omega^T S_w w \\ \tilde{S}_B = \omega^T S_B w \end{cases}$$

Linear Discriminant Analysis

~~LDA~~ for C > 2

- $$J(w) = \frac{|\tilde{S}_B|}{|\tilde{S}_W|} = \frac{|\omega^T S_B w|}{|\omega^T S_W w|}$$

Determinant

$$\frac{dJ}{dw} = 0 \rightarrow S_w^{-1} S_B w_i = \lambda_i w_i$$

$$\lambda_i = J(w_i)$$

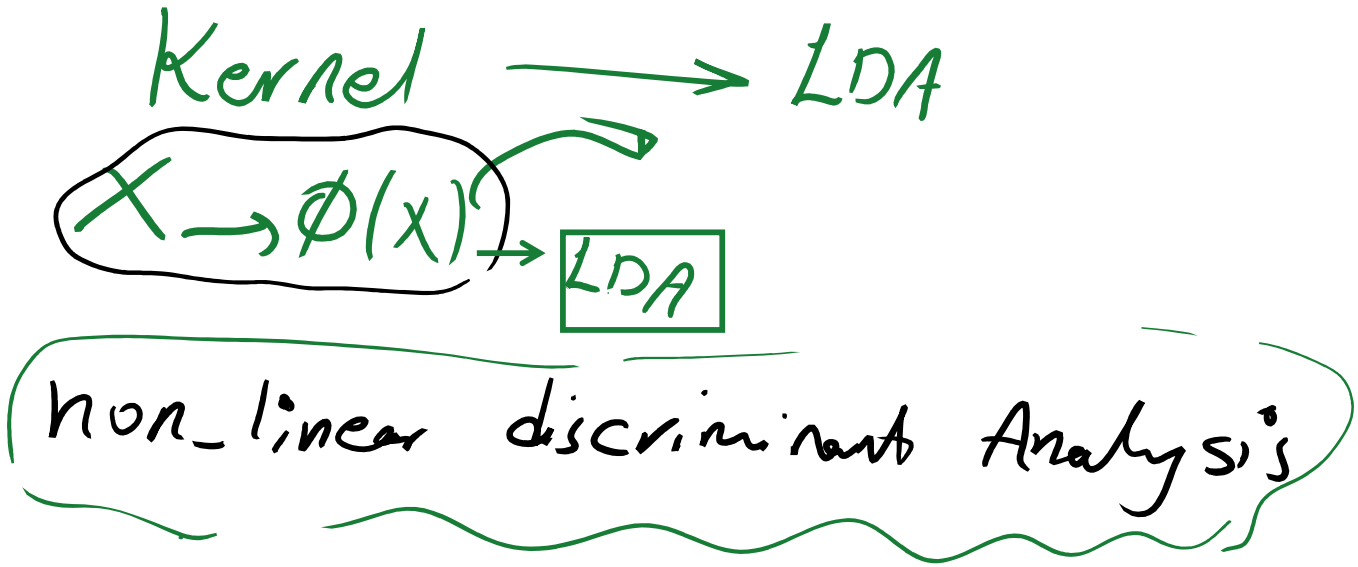
$$i = 1 \dots C-1$$

$W = [w_1 | w_2 | \dots | w_{C-1}]$ $n \times (C-1)$

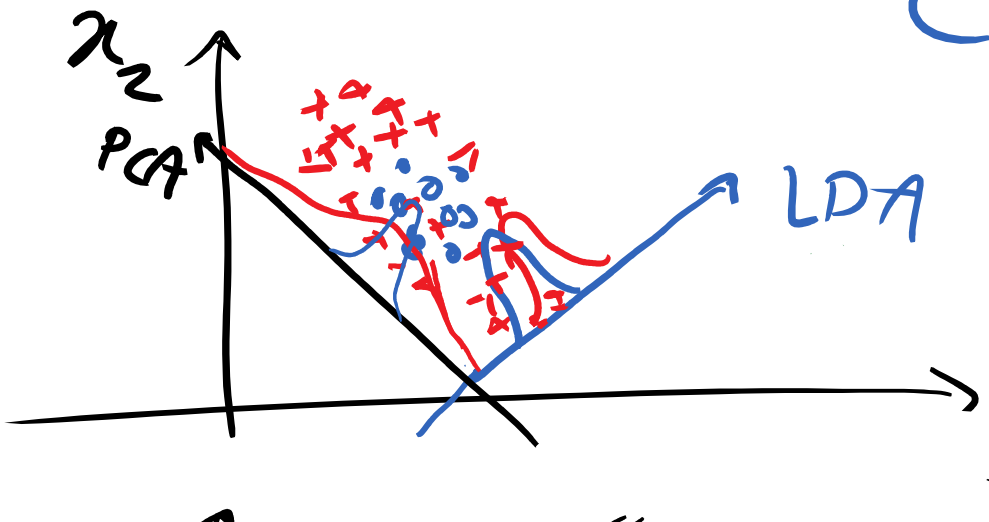
$C-1$ تا از بزرگ‌ترین مقادیر ویژه ماتریس $S_W^{-1} S_B$

w_i ها بردارهای ویژه متناظر با این مقادیر ویژه

KDA: kernel Discriminative analysis

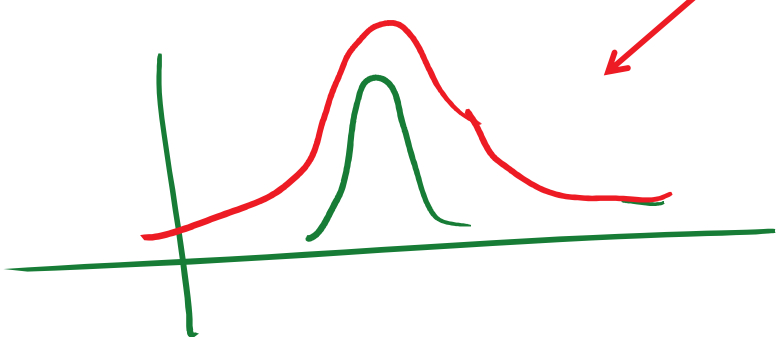


فرض LDA گوسی بودن توزیع نمونه‌های کلاسهاست.



حداکثر ماهیس خطی: 1-C

* درصورتی که اختلاف میانگین‌های کلاسها زیاد باشد LDA موفق نیست.



* Fisher LDA یا FLDA که در این جلسه بحث شده می‌شود

به عنوان روشی برای کاهش بعد حاصل از اینکه مدل مولد GDA استفاده

می‌شود به کار رود.

در حالتی که Σ کلاس‌ها برابر بود (LDA) اگر Σ برابر نبود ← QDA

* اگر ماتریس S_W، Singular بودن می‌توان ابتدا PCA اعمال کرد بعد LDA

GDA : $P(X \mid y_i)$

$y_i \in \{0, 1, 2, \ldots, k\}$

ایجاد مرز تصمیم خطی

LDA → GDA

LDA

$\Sigma_1 = \Sigma_2 = \cdots = \Sigma_k$

QDA : Σ متفاوت

مرز تصمیم مرتبه ۲